SQL for Data Engineering — Complete Handbook

from basics to modeling, loading patterns and performance — the data engineer's view

notes by Sagar · Jan 2026

Sagar

Contents

1. Where SQL fits in data engineering
2. Data types and table design
3. Querying fundamentals
4. NULL handling
5. Joins
6. Aggregation
7. Subqueries and CTEs
8. Set operations
9. Window functions
10. Dates and times
11. String functions and cleaning
12. Semi-structured data: JSON, arrays and structs
13. Loading data: DML and pipeline patterns
14. Data modeling for analytics
15. Views, materialized views and temporary objects
16. Transactions and consistency
17. Query performance
18. Data quality with SQL
19. Dialect cheat sheet
20. Pattern library

Sagar

1. Where SQL fits in data engineering

SQL is the language you use to shape, move and check data at every layer of a data platform. A data engineer uses it differently from an analyst. An analyst mostly reads data to answer questions. A data engineer writes SQL that builds tables, loads them every day, keeps them correct, and runs fast at scale.

Where you will write SQL

- **Warehouses:** Synapse dedicated pools, BigQuery, Snowflake, Redshift.
- **Lakehouse engines:** Spark SQL on Databricks, Hive on Hadoop, Trino/Presto, Athena.
- **Operational databases (sources):** PostgreSQL, MySQL, SQL Server, Oracle — you read from them to ingest data.
- **Transformation tools:** dbt models, Databricks SQL, stored procedures, ADF data flows that generate SQL.

What a data engineer uses SQL for

Job	Example
Build tables	**CREATE TABLE**, **CREATE TABLE AS SELECT**, views
Transform	Clean, join, aggregate raw data into silver and gold tables
Load	**INSERT**, **INSERT OVERWRITE**, **MERGE** for incremental and upsert loads
Keep history	SCD Type 2 dimensions
Check quality	Null, duplicate, range and reconciliation queries
Tune	Partitioning, clustering, reading query plans

ANSI SQL and dialects

ANSI SQL is the standard. Every engine supports most of it and then adds its own functions and syntax. The core — **SELECT**, joins, **GROUP BY**, window functions, CTEs — is almost identical everywhere. Differences show up in date functions, string functions, semi-structured data, and DDL. These notes use ANSI SQL by default and call out Spark SQL, BigQuery, Snowflake, PostgreSQL and T-SQL (SQL Server / Synapse) where they differ. Chapter 19 has a side-by-side cheat sheet.

> Write the logic in standard SQL first. Reach for dialect-specific functions only when they make the query simpler or faster.

2. Data types and table design

Choosing the right type is the first performance and correctness decision you make. A wrong type causes silent bugs (rounding money, losing time zones) and slow queries (joining strings instead of integers).

Common types

Category	Types	Notes
Integer	**TINYINT**, **SMALLINT**, **INT**, **BIGINT**	Use **BIGINT** for IDs that can grow large
Exact decimal	**DECIMAL(p, s)** / **NUMERIC**	Use for money. p = total digits, s = digits after the point
Approximate	**FLOAT**, **DOUBLE**	Fast, but rounding errors — never for money
Text	**VARCHAR(n)**, **STRING**, **TEXT**, **CHAR(n)**	Spark/BigQuery use **STRING** with no length
Boolean	**BOOLEAN**	T-SQL uses **BIT**
Date / time	**DATE**, **TIMESTAMP**, **TIMESTAMP WITH TIME ZONE**	Store event times in UTC
Binary	**BINARY**, **BYTES**, **VARBINARY**	Hashes, files
Complex	**ARRAY**, **STRUCT**, **MAP**, **JSON**, **VARIANT**	Lakehouse and cloud warehouses

Why DECIMAL for money

```
SELECT CAST(0.1 AS DOUBLE) + CAST(0.2 AS DOUBLE);      -- 0.30000000000000004
SELECT CAST(0.1 AS DECIMAL(10,2)) + CAST(0.2 AS DECIMAL(10,2));  -- 0.30
```

Floating point can't represent most decimal fractions exactly. Totals drift, and reconciliation with the source system fails by a few paise. Use **DECIMAL(18,2)** or wider for currency.

Timestamps and time zones

- Store event timestamps in **UTC**. Convert to local time only in the reporting layer.
- Know whether your engine's **TIMESTAMP** is "local" (no zone) or "instant" (zone-aware). Spark's **TIMESTAMP** is session-time-zone aware; **TIMESTAMP_NTZ** has no zone.
- Keep a separate **DATE** column for partitioning (for example **event_date**) derived from the UTC timestamp.

Creating tables

```
CREATE TABLE sales.orders (
    order_id      BIGINT        NOT NULL,
    customer_id   BIGINT        NOT NULL,
    order_ts      TIMESTAMP     NOT NULL,
    order_date    DATE          NOT NULL,
    amount        DECIMAL(18,2),
    status        VARCHAR(20),
    created_at    TIMESTAMP     DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT pk_orders PRIMARY KEY (order_id)
);
```

Constraints

Constraint	Meaning	In warehouses / lakehouse
NOT NULL	Column must have a value	Enforced almost everywhere, including Delta
PRIMARY KEY	Unique, not null identifier	Often **informational only** (not enforced) in BigQuery, Snowflake, Synapse, Databricks
FOREIGN KEY	Value must exist in parent table	Usually informational only
UNIQUE	No duplicates	Often not enforced
CHECK	Condition must be true	Enforced in Delta Lake, Postgres
DEFAULT	Value if none is given	Supported in most engines

> In analytical systems, uniqueness is usually **not enforced**. You must guarantee it in the pipeline — deduplicate before loading and run duplicate checks after. This is one of the biggest differences from OLTP databases.

Sagar

Create Table As Select (CTAS)

```
CREATE TABLE gold.daily_sales AS
SELECT order_date, SUM(amount) AS revenue, COUNT(*) AS orders
FROM silver.orders
GROUP BY order_date;
```

CTAS creates and fills a table in one step. It is the fastest way to materialize a transformation, and in many engines it is a single atomic operation.

Changing tables

```
ALTER TABLE sales.orders ADD COLUMN channel VARCHAR(20);
ALTER TABLE sales.orders RENAME COLUMN status TO order_status;
ALTER TABLE sales.orders ALTER COLUMN amount TYPE DECIMAL(20,2);   -- syntax varies
DROP TABLE IF EXISTS tmp.orders_stage;
TRUNCATE TABLE tmp.orders_stage;
```

Schema changes are a pipeline concern: adding a nullable column is usually safe; renaming or changing a type can break downstream jobs. Communicate schema changes before deploying them.

Managed vs external tables

- **Managed table:** the engine owns both metadata and data files. **DROP TABLE** deletes the data.
- **External table:** the engine stores only metadata and points to files you manage (for example in ADLS or S3). **DROP TABLE** removes the definition but leaves the files.

```
CREATE TABLE bronze.events (event_id STRING, payload STRING, event_date DATE)
USING PARQUET
PARTITIONED BY (event_date)
LOCATION 'abfss://lake@account.dfs.core.windows.net/bronze/events';
```

Sagar

3. Querying fundamentals

The basic shape

```
SELECT   customer_id, SUM(amount) AS total
FROM     orders
WHERE    order_date >= DATE '2026-01-01'
GROUP BY customer_id
HAVING   SUM(amount) > 10000
ORDER BY total DESC
LIMIT    10;
```

Logical order of execution

The order you write a query is not the order the engine evaluates it:

1. **FROM** and **JOIN** — build the row set
2. **WHERE** — filter rows
3. **GROUP BY** — form groups
4. **HAVING** — filter groups
5. **SELECT** — compute expressions and window functions
6. **DISTINCT**
7. **ORDER BY**
8. **LIMIT / OFFSET / TOP**

This explains several rules:

- A column alias defined in **SELECT** **can't be used in** **WHERE**, because **WHERE** runs first. It can be used in **ORDER BY**.
- Aggregates can't appear in **WHERE**; use **HAVING**.
- Window functions can't appear in **WHERE**; wrap the query or use **QUALIFY**.

Filtering

```
WHERE status = 'DELIVERED'
  AND amount BETWEEN 100 AND 500          -- inclusive on both ends
  AND city IN ('Bengaluru', 'Hyderabad')
  AND email LIKE '%@gmail.com'            -- % any chars, _ one char
  AND order_date >= DATE '2026-01-01'
  AND order_date <  DATE '2026-02-01'     -- half-open range
```

Use half-open ranges (**>= start AND < next_start**) for dates and timestamps. **BETWEEN '2026-01-01' AND '2026-01-31'** silently drops everything after midnight on the 31st when the column is a timestamp.

CASE expressions

```
SELECT order_id,
       CASE
         WHEN amount >= 10000 THEN 'high'
         WHEN amount >= 1000  THEN 'medium'
         ELSE 'low'
       END AS order_band
FROM orders;
```

CASE is evaluated top to bottom and stops at the first true condition. Without **ELSE**, unmatched rows get **NULL**.

DISTINCT

DISTINCT removes duplicate rows across **all** selected columns. It needs a full sort or shuffle, so on large data it is expensive. If you find yourself adding **DISTINCT** to "fix" duplicates, the real problem is usually a join that fans out — fix the join instead.

Sorting and limiting

- **ORDER BY** without **LIMIT** on a big table is costly and usually pointless in a pipeline — tables have no guaranteed order.
- Sort direction per column: **ORDER BY order_date DESC, order_id**.
- NULL ordering differs by engine; be explicit with **NULLS FIRST / NULLS LAST** where supported.
- Pagination: **LIMIT n OFFSET m** (ANSI-ish), **TOP n** (T-SQL), **FETCH FIRST n ROWS ONLY** (ANSI).

4. NULL handling

NULL means "unknown" or "missing". It is not zero and not an empty string. Most NULL bugs come from forgetting that SQL uses **three-valued logic**: TRUE, FALSE and UNKNOWN.

Comparisons with NULL

Expression	Result
`NULL = NULL`	UNKNOWN
`NULL <> 5`	UNKNOWN
`NULL IS NULL`	TRUE
`NULL + 10`	NULL
`'a' || NULL`	NULL (in most engines)

`WHERE` keeps only rows where the condition is TRUE. UNKNOWN rows are dropped, which causes this classic bug:

```
-- rows with city = NULL are silently excluded
SELECT * FROM customers WHERE city <> 'Delhi';

-- correct
SELECT * FROM customers WHERE city <> 'Delhi' OR city IS NULL;
```

NULL functions

```
COALESCE(phone, mobile, 'N/A')       -- first non-null value
NULLIF(denominator, 0)               -- NULL if equal, avoids divide-by-zero
CASE WHEN x IS NULL THEN 0 ELSE x END
IFNULL(x, 0) / ISNULL(x, 0) / NVL(x, 0)   -- dialect versions of 2-arg COALESCE
a IS DISTINCT FROM b                  -- NULL-safe "not equal"
a <=> b                              -- NULL-safe equal (Spark, MySQL)
```

NULLs in aggregates

- `COUNT(*)` counts rows. `COUNT(col)` skips NULLs.
- `SUM`, `AVG`, `MIN`, `MAX` ignore NULLs. `AVG` divides by non-null count only.
- `SUM` over zero rows or only NULLs returns NULL, not 0 — wrap with `COALESCE(SUM(x), 0)` if you need 0.

```
-- values: 10, 20, NULL
SELECT AVG(v)                -- 15  (30 / 2)
     , AVG(COALESCE(v, 0))   -- 10  (30 / 3)
FROM t;
```

Decide which one the business means. "Average order value" usually ignores missing orders; "average per day" may need zeros for empty days.

NULLs in joins, NOT IN and grouping

- NULL keys never match in a join (`NULL = NULL` is not true). Rows with NULL join keys drop out of inner joins.
- `x NOT IN (subquery)` returns no rows at all if the subquery contains a NULL. Use `NOT EXISTS`.
- `GROUP BY` puts all NULLs into one group.
- `UNION`, `INTERSECT`, `EXCEPT` and `DISTINCT` treat NULLs as equal to each other.

> When loading data, decide a NULL policy per column: reject the row, default it, or keep it as NULL. Document it, and test it.

5. Joins

Join types

Join	Returns
INNER JOIN	Rows with a match on both sides
LEFT JOIN	All left rows; right columns NULL when no match
RIGHT JOIN	All right rows; left columns NULL when no match
FULL OUTER JOIN	All rows from both sides
CROSS JOIN	Every combination (Cartesian product)
Semi join	Left rows that have a match (no right columns) — **EXISTS** / **LEFT SEMI JOIN**
Anti join	Left rows with no match — **NOT EXISTS** / **LEFT ANTI JOIN**

```
-- inner
SELECT o.order_id, c.name
FROM orders o
JOIN customers c ON c.customer_id = o.customer_id;

-- semi: customers who ordered
SELECT c.* FROM customers c
WHERE EXISTS (SELECT 1 FROM orders o WHERE o.customer_id = c.customer_id);

-- anti: customers who never ordered
SELECT c.* FROM customers c
WHERE NOT EXISTS (SELECT 1 FROM orders o WHERE o.customer_id = c.customer_id);

-- Spark SQL shortcuts
SELECT * FROM customers c LEFT SEMI JOIN orders o ON c.customer_id = o.customer_id;
SELECT * FROM customers c LEFT ANTI JOIN orders o ON c.customer_id = o.customer_id;
```

Fan-out: the most common join bug

If the join key is not unique on one side, rows multiply. Joining orders to a payments table that has two payment rows per order doubles those orders, and every **SUM(amount)** after that is wrong.

Always ask: **what is the grain of each table, and is my join key unique on the side I expect?**

```
-- check a key is unique before joining
SELECT customer_id, COUNT(*) FROM customers
GROUP BY customer_id HAVING COUNT(*) > 1;
```

Fixes: deduplicate the many side first, aggregate it to the right grain before joining, or pick one row per key with **ROW_NUMBER()**.

ON vs WHERE in outer joins

A filter on the right table placed in **WHERE** turns a **LEFT JOIN** into an inner join, because the NULL rows fail the filter.

```
-- WRONG: customers without a 2026 order disappear
SELECT c.customer_id, o.order_id
FROM customers c
LEFT JOIN orders o ON o.customer_id = c.customer_id
WHERE o.order_date >= DATE '2026-01-01';

-- RIGHT: filter inside ON keeps all customers
SELECT c.customer_id, o.order_id
FROM customers c
LEFT JOIN orders o ON o.customer_id = c.customer_id
                  AND o.order_date >= DATE '2026-01-01';
```

Self joins

The same table joined to itself, with different aliases — employee/manager, comparing a row to the previous period, finding pairs.

```
SELECT e.name AS employee, m.name AS manager
FROM emp e
LEFT JOIN emp m ON m.id = e.manager_id;
```

Non-equi and range joins

Join conditions don't have to be equality:

```
-- assign each sale to the price that was valid at that time
SELECT s.sale_id, p.price
FROM sales s
JOIN price_history p
  ON p.product_id = s.product_id
 AND s.sale_ts >= p.valid_from
 AND s.sale_ts <  COALESCE(p.valid_to, TIMESTAMP '9999-12-31 00:00:00');
```

Range joins are expensive at scale (they can't use hash joins on the range part). Keep an equality key in the condition whenever possible.

Join key hygiene

- Join on the same data type. Joining **INT** to **STRING** forces casts on every row and can produce wrong matches (**'007'** vs **7**).
- Trim and standardize case on text keys before loading, not inside every join.
- Prefer integer surrogate keys in the warehouse.

Sagar

6. Aggregation

Aggregate functions

```
COUNT(*), COUNT(col), COUNT(DISTINCT col)
SUM(x), AVG(x), MIN(x), MAX(x)
STDDEV(x), VARIANCE(x)
STRING_AGG(name, ', ')   -- LISTAGG (Snowflake), GROUP_CONCAT (MySQL), collect_list (Spark)
APPROX_COUNT_DISTINCT(user_id)   -- fast, ~1-2% error
PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY x)   -- median
```

GROUP BY rules

Every selected column must either be in **GROUP BY** or inside an aggregate. Group by the real key, not a display name — two customers can share a name.

```
SELECT customer_id, MAX(name) AS name, COUNT(*) AS orders
FROM orders
GROUP BY customer_id;
```

HAVING

HAVING filters after aggregation. Put row-level filters in **WHERE** so less data is grouped.

Conditional aggregation

One pass over the table, many metrics:

```
SELECT order_date,
       COUNT(*)                                                AS orders,
       SUM(CASE WHEN status = 'CANCELLED' THEN 1 ELSE 0 END) AS cancelled,
       SUM(CASE WHEN channel = 'APP' THEN amount ELSE 0 END) AS app_revenue,
       AVG(CASE WHEN status = 'DELIVERED' THEN amount END)   AS avg_delivered
FROM orders
GROUP BY order_date;
```

Postgres and Spark also support **COUNT(*) FILTER (WHERE status = 'CANCELLED')**.

Pivot and unpivot

```
-- pivot with CASE: works everywhere
SELECT product_id,
       SUM(CASE WHEN month = 1 THEN amount END) AS jan,
       SUM(CASE WHEN month = 2 THEN amount END) AS feb
FROM monthly_sales
GROUP BY product_id;

-- Spark / Snowflake / T-SQL also have PIVOT
SELECT * FROM monthly_sales
PIVOT (SUM(amount) FOR month IN (1 AS jan, 2 AS feb));

-- unpivot with UNION ALL (portable) or UNPIVOT / stack()
SELECT product_id, 'jan' AS month, jan AS amount FROM wide
UNION ALL
SELECT product_id, 'feb', feb FROM wide;
```

GROUPING SETS, ROLLUP, CUBE

Compute several grouping levels in one query — useful for report tables with subtotals.

```
SELECT region, city, SUM(amount) AS revenue
FROM sales
GROUP BY ROLLUP (region, city);
-- rows for (region, city), (region) subtotal, and () grand total

SELECT region, channel, SUM(amount)
FROM sales
GROUP BY GROUPING SETS ((region), (channel), ());

SELECT region, channel, SUM(amount)
FROM sales
GROUP BY CUBE (region, channel);   -- all combinations
```

GROUPING(col) returns 1 when the column is aggregated away in that row, so you can label subtotal rows.

Exact vs approximate distinct counts

COUNT(DISTINCT user_id) on billions of rows needs to shuffle every distinct value. **APPROX_COUNT_DISTINCT** (HyperLogLog) is many times cheaper, with a small error. Use approximate counts for dashboards and monitoring, exact counts for billing and finance.

Sagar

7. Subqueries and CTEs

Subquery types

```
-- scalar: returns one value
SELECT name, salary, salary - (SELECT AVG(salary) FROM emp) AS diff FROM emp;

-- derived table (inline view)
SELECT dept_id, avg_sal
FROM (SELECT dept_id, AVG(salary) AS avg_sal FROM emp GROUP BY dept_id) d
WHERE avg_sal > 50000;

-- IN / EXISTS
SELECT * FROM orders WHERE customer_id IN (SELECT customer_id FROM vip);

-- correlated: re-evaluated per outer row
SELECT e.name FROM emp e
WHERE e.salary > (SELECT AVG(salary) FROM emp x WHERE x.dept_id = e.dept_id);
```

Correlated subqueries are easy to read but can be slow. Modern optimizers often rewrite them as joins; when they don't, rewrite with a window function:

```
SELECT name
FROM (SELECT name, salary, AVG(salary) OVER (PARTITION BY dept_id) AS dept_avg FROM emp) t
WHERE salary > dept_avg;
```

Common Table Expressions

A CTE is a named subquery defined with **WITH**. It makes multi-step logic readable — each step does one thing.

```
WITH clean_orders AS (
    SELECT order_id, customer_id, CAST(amount AS DECIMAL(18,2)) AS amount, order_date
    FROM bronze.orders
    WHERE order_id IS NOT NULL
),
dedup AS (
    SELECT *, ROW_NUMBER() OVER (PARTITION BY order_id ORDER BY order_date DESC) AS rn
    FROM clean_orders
),
final AS (
    SELECT order_id, customer_id, amount, order_date FROM dedup WHERE rn = 1
)
SELECT * FROM final;
```

> A CTE is not automatically materialized. Some engines inline it everywhere it's referenced, so a CTE used three times may be computed three times. If an expensive step is reused, write it to a temp table or cache it.

Recursive CTEs

Used for hierarchies (org charts, category trees, bill of materials) and generating sequences.

```
WITH RECURSIVE org AS (
    SELECT id, name, manager_id, 1 AS lvl          -- anchor: top manager
    FROM emp WHERE manager_id IS NULL
    UNION ALL
    SELECT e.id, e.name, e.manager_id, o.lvl + 1   -- recursive step
    FROM emp e JOIN org o ON e.manager_id = o.id
)
SELECT * FROM org;
```

T-SQL omits the **RECURSIVE** keyword. Spark SQL added recursive CTEs only in recent versions — on older Spark, use iterative DataFrame logic or a fixed number of self joins. Always guard against cycles in real data.

8. Set operations

```
SELECT customer_id FROM online_orders
UNION           -- distinct rows from both
SELECT customer_id FROM store_orders;

SELECT customer_id FROM online_orders
UNION ALL       -- all rows, no dedup (faster)
SELECT customer_id FROM store_orders;

SELECT customer_id FROM online_orders
INTERSECT       -- in both
SELECT customer_id FROM store_orders;

SELECT customer_id FROM online_orders
EXCEPT          -- in first, not in second (MINUS in Oracle/Snowflake)
SELECT customer_id FROM store_orders;
```

Rules:

- Both queries must return the same number of columns with compatible types.
- Columns are matched **by position**, not by name. A swapped column order loads **city** into **name** without any error. Spark's DataFrame **unionByName** exists for this reason.
- Set operations treat NULLs as equal, unlike **=**.
- Use **UNION ALL** by default in pipelines; **UNION** adds a dedup step you rarely want.

EXCEPT is a quick way to compare two tables during testing:

```
(SELECT * FROM new_version EXCEPT SELECT * FROM old_version)
UNION ALL
(SELECT * FROM old_version EXCEPT SELECT * FROM new_version);
-- empty result = the two tables are identical
```

Sagar

9. Window functions

Window functions compute a value for each row using a set of related rows (the "window"), **without collapsing rows** like **GROUP BY** does. They are the most important SQL feature for data engineering.

Syntax

```
function(args) OVER (
    PARTITION BY col1, col2        -- groups (optional)
    ORDER BY col3                  -- order inside each group
    ROWS BETWEEN start AND end     -- frame (optional)
)
```

Ranking functions

Function	Ties	Example for 100, 100, 90
ROW_NUMBER()	Unique numbers, ties broken arbitrarily	1, 2, 3
RANK()	Ties share, then a gap	1, 1, 3
DENSE_RANK()	Ties share, no gap	1, 1, 2
NTILE(n)	Splits into n buckets	quartiles, deciles
PERCENT_RANK()	(rank − 1) / (rows − 1)	0 to 1
CUME_DIST()	Fraction of rows ≤ current	0 to 1

ROW_NUMBER() with a non-unique **ORDER BY** is non-deterministic: reruns can pick different rows. Always add a tie-breaker, like **ORDER BY updated_at DESC, ingestion_ts DESC, id**.

Offset functions

```
LAG(amount, 1, 0)  OVER (PARTITION BY customer_id ORDER BY order_date)  -- previous row (default 0)
LEAD(amount)       OVER (PARTITION BY customer_id ORDER BY order_date)  -- next row
FIRST_VALUE(amount) OVER w
LAST_VALUE(amount)  OVER w
NTH_VALUE(amount, 2) OVER w
```

Aggregate windows

```
SELECT order_id, customer_id, amount,
       SUM(amount)   OVER (PARTITION BY customer_id)                    AS customer_total,
       amount / SUM(amount) OVER (PARTITION BY customer_id)             AS share,
       AVG(amount)   OVER (PARTITION BY customer_id ORDER BY order_date
                           ROWS BETWEEN 2 PRECEDING AND CURRENT ROW)    AS avg_last_3,
       COUNT(*)      OVER ()                                            AS all_rows
FROM orders;
```

Frames: ROWS vs RANGE

The frame decides which rows inside the partition feed the calculation.

Frame	Meaning
ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW	Running total, row by row
ROWS BETWEEN 6 PRECEDING AND CURRENT ROW	Last 7 rows
ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING	Whole partition
RANGE BETWEEN INTERVAL 6 DAYS PRECEDING AND CURRENT ROW	Last 7 **days** by value (engine support varies)

Default frame rules (important):

- With **ORDER BY** and no frame: **RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW**. Rows with the same **ORDER BY** value are treated as one peer group, so a running total jumps by the whole group at once.
- Without **ORDER BY**: the whole partition.

That default is why **LAST_VALUE** surprises people — it returns the current row unless you widen the frame to **UNBOUNDED FOLLOWING**.

Sagar

QUALIFY

QUALIFY filters on window results the way **HAVING** filters on aggregates. Supported in Databricks/Spark SQL, Snowflake, BigQuery and Teradata.

```
SELECT *
FROM customer_updates
QUALIFY ROW_NUMBER() OVER (PARTITION BY customer_id ORDER BY updated_at DESC) = 1;
```

Without **QUALIFY**, wrap the query in a subquery or CTE and filter outside.

Named windows

```
SELECT order_id,
       SUM(amount) OVER w AS running,
       AVG(amount) OVER w AS running_avg
FROM orders
WINDOW w AS (PARTITION BY customer_id ORDER BY order_date
             ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW);
```

Performance notes

- Each distinct **PARTITION BY** / **ORDER BY** combination needs its own sort. Reuse the same window spec where possible.
- A window with no **PARTITION BY** sends all rows to one worker in distributed engines like Spark. Fine for small aggregated tables; dangerous on raw data.
- Skewed partition keys (one customer with millions of rows) create one slow task.

Sagar

10. Dates and times

Core operations (ANSI-style)

```
CURRENT_DATE, CURRENT_TIMESTAMP
EXTRACT(YEAR FROM order_ts), EXTRACT(MONTH FROM order_ts), EXTRACT(DOW FROM order_ts)
DATE_TRUNC('month', order_ts)          -- first moment of the month
order_date + INTERVAL '7' DAY
CAST(order_ts AS DATE)
```

Dialect differences

Operation	Spark SQL	BigQuery	Snowflake	T-SQL
Add days	`date_add(d, 7)`	`DATE_ADD(d, INTERVAL 7 DAY)`	`DATEADD(day, 7, d)`	`DATEADD(day, 7, d)`
Difference in days	`datediff(end, start)`	`DATE_DIFF(end, start, DAY)`	`DATEDIFF(day, start, end)`	`DATEDIFF(day, start, end)`
Truncate to month	`date_trunc('month', ts)` / `trunc(d,'MM')`	`DATE_TRUNC(d, MONTH)`	`DATE_TRUNC('month', d)`	`DATETRUNC(month, d)` (2022+)
Format	`date_format(ts, 'yyyy-MM-dd')`	`FORMAT_DATE('%Y-%m-%d', d)`	`TO_CHAR(d, 'YYYY-MM-DD')`	`FORMAT(d, 'yyyy-MM-dd')`
Parse	`to_date(s, 'dd-MM-yyyy')`	`PARSE_DATE('%d-%m-%Y', s)`	`TO_DATE(s, 'DD-MM-YYYY')`	`CONVERT(date, s, 105)`
Unix epoch → ts	`from_unixtime(sec)` / `timestamp_seconds(sec)`	`TIMESTAMP_SECONDS(sec)`	`TO_TIMESTAMP(sec)`	`DATEADD(s, sec, '1970-01-01')`

Argument order for date differences differs between engines. Spark's **`datediff(end, start)`** and Snowflake's **`DATEDIFF(day, start, end)`** look similar but are reversed. Test with known dates.

Time zones

```
-- Spark
from_utc_timestamp(event_ts_utc, 'Asia/Kolkata')
to_utc_timestamp(local_ts, 'Asia/Kolkata')
-- Snowflake / Postgres
CONVERT_TIMEZONE('UTC', 'Asia/Kolkata', event_ts)
event_ts AT TIME ZONE 'Asia/Kolkata'
```

Store UTC, convert at the edge, and use named zones (**`Asia/Kolkata`**, **`America/New_York`**) rather than fixed offsets, so daylight saving is handled.

Calendar (date dimension) table

A calendar table with one row per day removes most date headaches: fiscal periods, holidays, week numbers and gap filling.

```
-- Spark SQL
SELECT d AS date,
       year(d) AS year, quarter(d) AS quarter, month(d) AS month,
       date_format(d, 'EEEE') AS day_name,
       dayofweek(d) IN (1, 7) AS is_weekend
FROM (SELECT explode(sequence(DATE '2020-01-01', DATE '2030-12-31', INTERVAL 1 DAY)) AS d);

-- BigQuery
SELECT d FROM UNNEST(GENERATE_DATE_ARRAY('2020-01-01', '2030-12-31')) AS d;

-- Postgres
SELECT generate_series(DATE '2020-01-01', DATE '2030-12-31', INTERVAL '1 day')::date AS d;
```

Join facts to the calendar with a **`LEFT JOIN`** from the calendar side to fill days with no activity.

II. String functions and cleaning

Common functions

```
UPPER(s), LOWER(s), INITCAP(s)
TRIM(s), LTRIM(s), RTRIM(s)
LENGTH(s)                       -- LEN in T-SQL
SUBSTRING(s, start, len)        -- 1-based in SQL
CONCAT(a, b), a || b            -- + in T-SQL
CONCAT_WS('-', a, b, c)         -- skips NULLs
REPLACE(s, 'old', 'new')
LPAD(id, 8, '0'), RPAD(...)
POSITION('x' IN s) / INSTR(s, 'x') / CHARINDEX('x', s)
SPLIT(s, ',')                   -- returns array (Spark, BigQuery); SPLIT_PART(s, ',', 2) in Snowflake/Postgres
```

Regular expressions

```
-- Spark SQL
regexp_extract(email, '@(.+)$', 1)              -- domain
regexp_replace(phone, '[^0-9]', '')             -- keep digits
email RLIKE '^[^@]+@[^@]+\\.[a-z]{2,}$'         -- validate
-- BigQuery: REGEXP_EXTRACT, REGEXP_REPLACE, REGEXP_CONTAINS
-- Snowflake: REGEXP_SUBSTR, REGEXP_REPLACE, RLIKE
```

A standard cleaning step

```
SELECT
  CAST(TRIM(customer_id) AS BIGINT)                    AS customer_id,
  INITCAP(TRIM(name))                                  AS name,
  LOWER(TRIM(email))                                   AS email,
  NULLIF(regexp_replace(phone, '[^0-9]', ''), '')      AS phone,
  CASE WHEN UPPER(TRIM(country)) IN ('IN', 'IND', 'INDIA') THEN 'IN'
       ELSE UPPER(TRIM(country)) END                   AS country_code
FROM bronze.customers;
```

Clean once in the silver layer so every downstream query doesn't repeat it. Turn empty strings into NULL so "missing" means one thing.

12. Semi-structured data: JSON, arrays and structs

Raw data from APIs, Kafka and logs often arrives as JSON. Modern engines can query it directly.

JSON strings

```
-- Spark SQL
get_json_object(payload, '$.customer.id')
from_json(payload, 'id INT, items ARRAY<STRUCT<sku: STRING, qty: INT>>')
schema_of_json('{"id":1,"tags":["a"]}')
-- BigQuery
JSON_VALUE(payload, '$.customer.id'), JSON_QUERY(payload, '$.items')
-- Snowflake (VARIANT column)
payload:customer.id::NUMBER
-- Postgres (jsonb)
payload->'customer'->>'id'
-- T-SQL / Synapse
JSON_VALUE(payload, '$.customer.id'), OPENJSON(payload)
```

Structs and arrays

```
-- struct field access
SELECT customer.name, customer.address.city FROM orders;

-- array functions (Spark)
size(items), array_contains(tags, 'vip'), element_at(items, 1),
array_distinct(tags), sort_array(tags), transform(items, x -> x.qty * 2),
filter(items, x -> x.qty > 0), aggregate(items, 0, (acc, x) -> acc + x.qty)
```

Exploding arrays into rows

```
-- Spark / Hive
SELECT o.order_id, i.sku, i.qty
FROM orders o
LATERAL VIEW EXPLODE(o.items) t AS i;
-- or
SELECT order_id, item.sku, item.qty
FROM (SELECT order_id, explode(items) AS item FROM orders);

-- keep orders with empty or NULL arrays
LATERAL VIEW OUTER EXPLODE(o.items) t AS i

-- BigQuery
SELECT o.order_id, i.sku FROM orders o, UNNEST(o.items) AS i;

-- Snowflake
SELECT o.order_id, f.value:sku::STRING
FROM orders o, LATERAL FLATTEN(input => o.items) f;
```

Building nested data

```
-- Spark: one row per order with an array of items
SELECT order_id, collect_list(named_struct('sku', sku, 'qty', qty)) AS items
FROM order_items GROUP BY order_id;
```

Parse JSON once with an explicit schema in the bronze-to-silver step and store typed columns. Repeatedly parsing JSON strings in every query is slow and hides schema drift.

Sagar

13. Loading data: DML and pipeline patterns

INSERT

```
INSERT INTO silver.orders (order_id, customer_id, amount, order_date)
SELECT order_id, customer_id, amount, order_date FROM staging.orders;
```

Always list the columns. **INSERT INTO t SELECT *** breaks or loads the wrong columns as soon as a column is added or reordered.

INSERT OVERWRITE (partition replace)

```
-- Spark / Hive: replace only the partitions in the result
SET spark.sql.sources.partitionOverwriteMode = dynamic;
INSERT OVERWRITE TABLE silver.orders PARTITION (order_date)
SELECT order_id, customer_id, amount, order_date
FROM staging.orders
WHERE order_date = DATE '2026-01-15';
```

With static mode (the default), **INSERT OVERWRITE** on a partitioned table without a partition spec can wipe the whole table. With dynamic mode, only partitions present in the query output are replaced. Delta Lake uses **replaceWhere** for the same purpose.

UPDATE and DELETE

```
UPDATE silver.customers SET city = 'Bengaluru' WHERE city = 'Bangalore';
DELETE FROM silver.orders WHERE order_date < DATE '2020-01-01';
```

On Delta, Iceberg, BigQuery and Snowflake these rewrite the affected files or micro-partitions. Running many small updates is expensive — batch them.

MERGE (upsert)

```
MERGE INTO silver.customers AS t
USING staging.customers AS s
ON t.customer_id = s.customer_id
WHEN MATCHED AND s.op = 'D' THEN DELETE
WHEN MATCHED AND (t.name <> s.name OR t.city <> s.city) THEN
    UPDATE SET name = s.name, city = s.city, updated_at = s.updated_at
WHEN NOT MATCHED AND s.op <> 'D' THEN
    INSERT (customer_id, name, city, updated_at)
    VALUES (s.customer_id, s.name, s.city, s.updated_at);
```

Rules that matter:

- The source must have **at most one row per key**. Deduplicate it first, or MERGE fails (or behaves unpredictably on some engines).
- Add a change condition to **WHEN MATCHED** so unchanged rows aren't rewritten.
- NULL-safe comparisons for change detection: **t.city IS DISTINCT FROM s.city**, or compare a row hash.
- Include the partition column in the **ON** condition when you can, so the engine only scans relevant partitions.

Full load vs incremental load

Pattern	How	Use when
Full refresh	**TRUNCATE** + **INSERT**, or **CREATE OR REPLACE TABLE AS**	Small tables, or the source can't tell you what changed
Append	**INSERT** new rows only	Immutable event data (clicks, logs)
Incremental by watermark	Filter source on **updated_at > last_watermark**, then MERGE	Source has a reliable updated timestamp
Partition overwrite	Rebuild only affected date partitions	Daily facts where late data lands in recent days
CDC	Apply insert/update/delete events in order	Source emits change logs (Debezium, Delta change data feed)

Sagar

Watermark-based incremental load

```
-- 1. read the last watermark from a control table
SELECT last_value FROM etl.watermarks WHERE table_name = 'orders';

-- 2. stage only changed rows (with a small overlap for late commits)
CREATE OR REPLACE TEMP VIEW inc AS
SELECT * FROM source.orders
WHERE updated_at > TIMESTAMP '<last_value>' - INTERVAL 1 HOUR;

-- 3. dedup + MERGE inc into target (overlap rows just re-match)

-- 4. advance the watermark only after the MERGE succeeds
UPDATE etl.watermarks
SET last_value = (SELECT MAX(updated_at) FROM inc)
WHERE table_name = 'orders';
```

Idempotency

A load is **idempotent** if running it twice gives the same result as running it once. This is the property that makes reruns and backfills safe.

- Idempotent: MERGE on a key, partition overwrite, **CREATE OR REPLACE**.
- Not idempotent: plain **INSERT** of the same batch (duplicates), **UPDATE x = x + 1**.

> Design every pipeline step to be rerunnable. When a job fails at 3 AM, you want "just rerun it" to be the correct answer.

SCD Type 2 with SQL

```
-- step 1: close current rows whose tracked attributes changed
MERGE INTO dim_customer t
USING (SELECT * FROM staging_customers) s
ON t.customer_id = s.customer_id AND t.is_current = TRUE
WHEN MATCHED AND (t.city IS DISTINCT FROM s.city OR t.segment IS DISTINCT FROM s.segment) THEN
  UPDATE SET is_current = FALSE, valid_to = s.load_ts;

-- step 2: insert a new current row for new and changed customers
INSERT INTO dim_customer (customer_sk, customer_id, city, segment, valid_from, valid_to, is_current)
SELECT <surrogate key>, s.customer_id, s.city, s.segment, s.load_ts, NULL, TRUE
FROM staging_customers s
LEFT JOIN dim_customer t
  ON t.customer_id = s.customer_id AND t.is_current = TRUE
WHERE t.customer_id IS NULL;
```

After step 1, changed customers no longer have a current row, so step 2 inserts them along with brand-new customers. Run both steps in one transaction where the engine supports it. A single-MERGE version exists using a "union of changed rows with a NULL merge key" trick; the two-step version is easier to read and debug.

14. Data modeling for analytics

OLTP vs OLAP

	OLTP (source systems)	OLAP (warehouse / lakehouse)
Purpose	Run the business: orders, payments	Analyze the business
Queries	Small reads/writes by key	Large scans and aggregations
Design	Normalized (3NF)	Denormalized (star schema)
Storage	Row-oriented	Column-oriented
Constraints	Enforced	Mostly informational

Normalization (for reading source systems)

- **1NF:** atomic values, no repeating groups.
- **2NF:** every non-key column depends on the whole key.
- **3NF:** no column depends on another non-key column.

Normalization removes duplication and update anomalies. Analytics prefers denormalized tables because joins at scale are expensive and analysts need simple models.

Dimensional modeling (Kimball)

- **Fact table:** measurable events — orders, payments, page views. Contains foreign keys to dimensions plus numeric measures.
- **Dimension table:** descriptive context — customer, product, date, store.
- **Star schema:** fact in the middle, dimensions around it, one join from fact to each dimension.
- **Snowflake schema:** dimensions normalized into sub-dimensions (product → category). Saves space, adds joins.

Grain: decide it first

The grain is what one row of the fact table represents: "one row per order line", "one row per customer per day". Declare it before choosing columns. Mixing grains in one fact table is the root of most double-counting bugs.

Types of fact tables

Type	Grain	Example
Transaction	One row per event	Order lines, payments
Periodic snapshot	One row per entity per period	Daily inventory per store, month-end balances
Accumulating snapshot	One row per process instance, updated as it progresses	Order lifecycle: placed, shipped, delivered dates
Factless	Events with no measure	Student attendance, promotion eligibility

Measures

- **Additive:** can be summed across all dimensions (amount, quantity).
- **Semi-additive:** can be summed across some dimensions but not time (account balance, inventory level — use the last value or average over time).
- **Non-additive:** ratios and percentages — store the numerator and denominator, compute the ratio at query time.

Keys

- **Natural / business key:** from the source (**customer_code**). Can change, can collide across systems.
- **Surrogate key:** warehouse-generated integer or hash. Stable, compact, needed for SCD Type 2 (one customer → many versions).

```
-- hash surrogate key (deterministic, no sequence needed)
SELECT sha2(concat_ws('||', source_system, customer_code), 256) AS customer_sk, ...
-- or identity columns: GENERATED ALWAYS AS IDENTITY (Delta, Snowflake, Postgres)
```

Slowly Changing Dimensions

Type	Behaviour	Use for
0	Never change	Date of birth, original signup date
1	Overwrite	Corrections, attributes where history doesn't matter
2	New row per change, with **valid_from**, **valid_to**, **is_current**	Attributes where reports must reflect the value at the time (customer segment, region)
3	Keep previous value in an extra column	Only the last change matters
6	1 + 2 + 3 combined	Both history and current value on every row

Facts join to the SCD2 version that was valid at event time:

```
SELECT f.order_id, d.segment
FROM fact_orders f
JOIN dim_customer d
  ON d.customer_id = f.customer_id
 AND f.order_ts >= d.valid_from
 AND (f.order_ts < d.valid_to OR d.valid_to IS NULL);
```

Better: resolve the surrogate key during the fact load and store **customer_sk** on the fact — then the join is a simple equality.

Other modeling tools

- **Conformed dimensions:** one shared **dim_date**, **dim_customer** used by every fact, so metrics line up across subject areas.
- **Junk dimension:** combine small flags (is_gift, payment_type) into one dimension instead of many tiny ones.
- **Degenerate dimension:** a transaction ID kept on the fact with no dimension table (order number).
- **Bridge table:** resolve many-to-many relationships (accounts with multiple owners).
- **One Big Table (OBT):** a wide, fully denormalized table for a specific use. Fast and simple to query, but duplicated logic and harder to maintain.
- **Data Vault:** hubs (keys), links (relationships), satellites (attributes with history). Good for auditability and many changing sources; usually feeds a star schema for reporting.

Medallion layers

Layer	Content	SQL work
Bronze	Raw data as received, plus ingestion metadata	Append, minimal parsing
Silver	Cleaned, typed, deduplicated, conformed entities	Cleaning, dedup, MERGE, SCD
Gold	Business-level facts, dimensions and aggregates	Star schemas, KPIs, reporting tables

15. Views, materialized views and temporary objects

Views

A view is a saved query. It stores no data; every read runs the query.

```
CREATE OR REPLACE VIEW gold.v_active_customers AS
SELECT customer_id, name, city
FROM silver.customers
WHERE is_active = TRUE;
```

Uses: hide complexity, expose only some columns (security), keep a stable interface while the underlying tables change.

Materialized views

A materialized view stores the query result and refreshes it (automatically or on schedule). Reads are fast; storage and refresh cost money. Supported in BigQuery, Snowflake, Synapse, Databricks SQL and Postgres, each with limits on what queries are allowed.

Temporary objects

```
CREATE TEMPORARY VIEW stage_v AS SELECT ...;      -- Spark: session-scoped
CREATE TEMP TABLE stage_t AS SELECT ...;          -- Postgres / Snowflake
SELECT ... INTO #stage FROM ...;                  -- T-SQL temp table
```

Use temp objects to break a big transformation into steps, or to compute an expensive intermediate result once and reuse it.

Choosing

Need	Use
Simple reusable logic, always fresh	View
Expensive query read many times	Materialized view or a table built by the pipeline
Intermediate step inside one job	Temp view / temp table / CTE
Full control over refresh and history	A pipeline-managed table

16. Transactions and consistency

ACID

- **Atomicity:** all of a transaction happens, or none.
- **Consistency:** data moves from one valid state to another.
- **Isolation:** concurrent transactions don't see each other's partial work.
- **Durability:** committed data survives failures.

```
BEGIN TRANSACTION;
UPDATE accounts SET balance = balance - 500 WHERE id = 1;
UPDATE accounts SET balance = balance + 500 WHERE id = 2;
COMMIT;       -- or ROLLBACK on error
```

Isolation levels (OLTP sources)

Level	Prevents
Read uncommitted	Nothing — dirty reads possible
Read committed	Dirty reads
Repeatable read	Dirty + non-repeatable reads
Serializable	Dirty, non-repeatable and phantom reads

When you extract from a busy OLTP database, read from a replica or snapshot so long extraction queries don't block the application, and so the extract is consistent.

In the lakehouse

Plain Parquet files on a data lake have **no transactions**: a failed job can leave half-written files, and readers can see partial results. Table formats — Delta Lake, Apache Iceberg, Apache Hudi — add a transaction log that gives ACID commits, so a write is either fully visible or not at all. They use optimistic concurrency: two writers that touch the same files conflict, and one retries.

17. Query performance

Performance in analytics comes from **reading less data** and **moving less data between machines**.

Read the plan first

```
EXPLAIN SELECT ...;              -- Postgres, Spark, Snowflake, Synapse
EXPLAIN ANALYZE SELECT ...;      -- Postgres: runs it and shows actual times
EXPLAIN FORMATTED SELECT ...;    -- Spark: readable physical plan
```

BigQuery shows an execution graph per query; Snowflake has the query profile; Databricks has the Spark UI and query profile. Look for: full scans, huge shuffles/exchanges, the join strategy, skewed tasks, and spills to disk.

Storage layout

Technique	What it does	Where
Partitioning	Splits data into folders/segments by a column (usually date). Filters on it skip whole partitions.	Hive, Spark, Delta, BigQuery, Synapse
Clustering / Z-order / liquid clustering	Co-locates similar values inside files so min/max stats skip files	BigQuery, Snowflake, Delta
Bucketing	Pre-hashes rows into a fixed number of buckets by key to avoid shuffles in joins	Hive, Spark
Indexes	B-tree lookups by key	OLTP databases, Synapse, Postgres
Distribution	How rows spread across nodes (hash, round-robin, replicated)	Synapse dedicated pools, Redshift
Columnar formats	Read only needed columns, compressed	Parquet, ORC, warehouse storage

Partitioning rules of thumb:

- Partition by a column used in most filters, with **low to moderate cardinality** (date, not customer_id).
- Too many small partitions is as bad as none — aim for partitions large enough to hold reasonably sized files.

Write queries that can be pruned

```
-- prevents pruning: function on the partition column
WHERE YEAR(order_date) = 2026
WHERE CAST(order_date AS STRING) LIKE '2026%'

-- allows pruning
WHERE order_date >= DATE '2026-01-01' AND order_date < DATE '2027-01-01'
```

General rules

- Select only the columns you need. **SELECT** * on a columnar table reads every column.
- Filter early, in **WHERE**, before joins and aggregations.
- Aggregate before joining when the join would fan out.
- Join on integer keys of matching types.
- Broadcast small tables in distributed engines (**/*+ BROADCAST(dim) */** in Spark).
- Watch for skew: one key with most of the rows makes one task run far longer than the rest.
- Avoid **DISTINCT** and **ORDER BY** you don't need.
- Prefer **UNION ALL** over **UNION**, **EXISTS** over **IN** with big subqueries, window functions over self joins.
- Use approximate functions for monitoring and exploration.
- Keep statistics fresh (**ANALYZE TABLE ... COMPUTE STATISTICS** in Spark/Hive) so the optimizer picks good plans.
- Fix the small-file problem: compact files (**OPTIMIZE** in Delta).

Cost awareness in cloud warehouses

- **BigQuery (on-demand):** you pay for bytes scanned. Partition + cluster tables, set **require_partition_filter**, avoid **SELECT** *, preview with dry runs.
- **Snowflake:** you pay for warehouse running time. Right-size warehouses, auto-suspend, use clustering keys on very large tables.
- **Synapse dedicated / Databricks:** you pay for compute time. Faster queries are cheaper queries.

Sagar

18. Data quality with SQL

Quality checks are SQL queries that return rows (or counts) when something is wrong. Run them in the pipeline and fail or alert on bad results.

Core checks

```
-- duplicates on the business key
SELECT order_id, COUNT(*) FROM silver.orders
GROUP BY order_id HAVING COUNT(*) > 1;

-- nulls in mandatory columns
SELECT COUNT(*) AS bad_rows FROM silver.orders
WHERE order_id IS NULL OR customer_id IS NULL OR order_date IS NULL;

-- valid values
SELECT status, COUNT(*) FROM silver.orders
WHERE status NOT IN ('PLACED', 'SHIPPED', 'DELIVERED', 'CANCELLED')
GROUP BY status;

-- ranges
SELECT COUNT(*) FROM silver.orders WHERE amount < 0 OR amount > 10000000;

-- referential integrity (orphans)
SELECT COUNT(*) FROM fact_orders f
LEFT JOIN dim_customer d ON d.customer_sk = f.customer_sk
WHERE d.customer_sk IS NULL;

-- freshness
SELECT MAX(order_ts) AS latest FROM silver.orders;   -- alert if older than SLA
```

Volume anomaly check

```
WITH daily AS (
  SELECT order_date, COUNT(*) AS cnt
  FROM silver.orders
  WHERE order_date >= CURRENT_DATE - INTERVAL 31 DAYS
  GROUP BY order_date
),
stats AS (
  SELECT AVG(cnt) AS mu, STDDEV(cnt) AS sd
  FROM daily WHERE order_date < CURRENT_DATE - INTERVAL 1 DAY
)
SELECT d.order_date, d.cnt
FROM daily d CROSS JOIN stats s
WHERE d.order_date = CURRENT_DATE - INTERVAL 1 DAY
  AND ABS(d.cnt - s.mu) > 3 * s.sd;
```

Reconciliation (source vs target)

```
SELECT 'source' AS side, COUNT(*) AS rows, SUM(amount) AS total FROM source.orders WHERE order_date = DATE '2026-01-15'
UNION ALL
SELECT 'target', COUNT(*), SUM(amount) FROM silver.orders WHERE order_date = DATE '2026-01-15';
```

Compare counts **and** sums of key measures. Counts can match while amounts don't.

Row-level comparison with a hash

```
SELECT s.order_id
FROM source_snapshot s
JOIN target_snapshot t ON t.order_id = s.order_id
WHERE sha2(concat_ws('|', s.amount, s.status, s.customer_id), 256)
   <> sha2(concat_ws('|', t.amount, t.status, t.customer_id), 256);
```

`concat_ws` skips NULLs, so **`('a', NULL)`** and **`(NULL, 'a')`** can produce the same string — wrap columns in **`COALESCE(CAST(col AS STRING), '~')`** first.

Sagar

Blocking vs warning checks

- **Blocking:** *primary-key duplicates, nulls in keys, schema mismatch → fail the load.*
- **Warning:** *volume anomalies, unusual distributions → alert a human, keep loading.*

Sagar

19. Dialect cheat sheet

Task	Spark SQL / Databricks	BigQuery	Snowflake	PostgreSQL	T-SQL (SQL Server / Synapse)
Limit rows	`LIMIT n`	`LIMIT n`	`LIMIT n`	`LIMIT n`	`TOP n`
String concat	`concat`, `||`	`CONCAT`, `||`	`||`	`||`	`+`, `CONCAT`
If null	`coalesce`, `nvl`	`IFNULL`, `COALESCE`	`NVL`, `COALESCE`	`COALESCE`	`ISNULL`, `COALESCE`
Current ts	`current_timestamp()`	`CURRENT_TIMESTAMP()`	`CURRENT_TIMESTAMP`	`now()`	`GETDATE()`, `SYSDATETIME()`
Filter on window	`QUALIFY`	`QUALIFY`	`QUALIFY`	subquery	subquery
Upsert	`MERGE INTO` (Delta)	`MERGE`	`MERGE`	`INSERT ... ON CONFLICT`, `MERGE` (15+)	`MERGE`
Explode array	`explode`, `LATERAL VIEW`	`UNNEST`	`LATERAL FLATTEN`	`unnest`	`OPENJSON`, `CROSS APPLY`
Regex extract	`regexp_extract`	`REGEXP_EXTRACT`	`REGEXP_SUBSTR`	`substring(s from 'pat')`	limited (`PATINDEX`)
Approx distinct	`approx_count_distinct`	`APPROX_COUNT_DISTINCT`	`APPROX_COUNT_DISTINCT`	extension	`APPROX_COUNT_DISTINCT`
Create or replace	`CREATE OR REPLACE TABLE`	`CREATE OR REPLACE TABLE`	`CREATE OR REPLACE TABLE`	`DROP + CREATE`	`CREATE OR ALTER` (views/procs)
Set difference	`EXCEPT`	`EXCEPT DISTINCT`	`EXCEPT / MINUS`	`EXCEPT`	`EXCEPT`
Generate dates	`sequence + explode`	`GENERATE_DATE_ARRAY`	`GENERATOR + DATEADD`	`generate_series`	recursive CTE

Sagar

20. Pattern library

Reusable solutions to problems you'll meet in real pipelines.

Latest record per key

```
SELECT * FROM (
  SELECT *, ROW_NUMBER() OVER (PARTITION BY id ORDER BY updated_at DESC, ingest_ts DESC) AS rn
  FROM bronze.customers
) t WHERE rn = 1;
```

Remove exact duplicates

```
SELECT DISTINCT * FROM bronze.events;          -- small/medium data
-- or keep one per full-row hash with ROW_NUMBER for control over which survives
```

Change detection between snapshots

```
SELECT COALESCE(n.id, o.id) AS id,
       CASE WHEN o.id IS NULL THEN 'INSERT'
            WHEN n.id IS NULL THEN 'DELETE'
            WHEN n.row_hash <> o.row_hash THEN 'UPDATE'
       END AS change_type
FROM today n
FULL OUTER JOIN yesterday o ON o.id = n.id
WHERE o.id IS NULL OR n.id IS NULL OR n.row_hash <> o.row_hash;
```

Running totals and moving averages

```
SUM(amount) OVER (PARTITION BY store_id ORDER BY sale_date ROWS UNBOUNDED PRECEDING)
AVG(amount) OVER (PARTITION BY store_id ORDER BY sale_date ROWS BETWEEN 6 PRECEDING AND CURRENT ROW)
```

Fill missing days

```
SELECT c.date, s.store_id, COALESCE(f.amount, 0) AS amount
FROM dim_date c
CROSS JOIN (SELECT DISTINCT store_id FROM sales) s
LEFT JOIN sales f ON f.sale_date = c.date AND f.store_id = s.store_id
WHERE c.date BETWEEN DATE '2026-01-01' AND DATE '2026-01-31';
```

Gaps and islands (streaks)

```
WITH d AS (SELECT DISTINCT user_id, login_date FROM logins),
g AS (
  SELECT user_id, login_date,
         date_sub(login_date, ROW_NUMBER() OVER (PARTITION BY user_id ORDER BY login_date)) AS grp
  FROM d)
SELECT user_id, MIN(login_date) AS streak_start, MAX(login_date) AS streak_end, COUNT(*) AS days
FROM g GROUP BY user_id, grp;
```

Sessionization

```
WITH x AS (
  SELECT *, CASE WHEN unix_timestamp(ts) - unix_timestamp(LAG(ts) OVER (PARTITION BY user_id ORDER BY ts)) > 1800
                   OR LAG(ts) OVER (PARTITION BY user_id ORDER BY ts) IS NULL
              THEN 1 ELSE 0 END AS new_session
  FROM clicks)
SELECT *, SUM(new_session) OVER (PARTITION BY user_id ORDER BY ts) AS session_no FROM x;
```

Forward fill the last known value

```
SELECT device_id, ts,
       LAST_VALUE(temp, TRUE) OVER (PARTITION BY device_id ORDER BY ts
                                    ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) AS temp_filled
FROM readings;
-- Spark: last(temp, true); Snowflake/BigQuery: LAST_VALUE(temp IGNORE NULLS)
```

Top N per group

```
SELECT * FROM (
  SELECT *, DENSE_RANK() OVER (PARTITION BY category ORDER BY revenue DESC) AS r
  FROM product_revenue) t
WHERE r <= 5;
```

Percent of total

```
SELECT category, revenue, 100.0 * revenue / SUM(revenue) OVER () AS pct FROM category_revenue;
```

Point-in-time (as-of) join

```
SELECT f.*, d.segment
FROM fact_orders f
JOIN dim_customer d
  ON d.customer_id = f.customer_id
 AND f.order_ts >= d.valid_from
 AND f.order_ts < COALESCE(d.valid_to, TIMESTAMP '9999-12-31 00:00:00');
```

Year-over-year comparison

```
WITH m AS (SELECT date_trunc('month', order_date) AS month, SUM(amount) AS rev FROM orders GROUP BY 1)
SELECT cur.month, cur.rev, prev.rev AS rev_last_year,
       100.0 * (cur.rev - prev.rev) / NULLIF(prev.rev, 0) AS yoy_pct
FROM m cur
LEFT JOIN m prev ON prev.month = add_months(cur.month, -12);
```

Keep this chapter as your toolbox. Most real transformation work is a combination of these patterns, applied at the right grain, with the edge cases (NULLs, duplicates, late data) handled.

— **Sagar**

Sagar